BAIN & COMPANY

An Architecture for Agentic AI

The Three Layers of an Agentic AI Platform

Modern AI platforms are converging around a shared architecture built for orchestration, visibility, and governed data access.

By Eric Sheng, Roger Zhu, Brendan O'Rourke, Dale Pedzinski, and Kevin Zhang

5 min read

At a Glance

- Legacy enterprise AI platforms are structurally mismatched for agentic systems.
- Modern agentic AI demands a three-layer architecture centered on orchestration, observability, and governed data access.
- Security and governance must be embedded by design, not bolted on after deployment.

This is part two of a four-part series on architecting for agentic AI.

In the first post in this series, we made the case that agentic AI demands a fundamentally new architecture built for connected, nondeterministic systems rather than isolated models. Here, we discuss what that architecture looks like.

Agentic architecture converges around orchestration and governance

Most enterprise AI platforms were built for simpler systems: single models serving narrow use cases behind static application programming interface (API) endpoints. Data flowed into specific applications through fixed extract, transform, load (ETL) pipelines; orchestration logic

lived within individual systems; and governance was typically tacked on after deployment.

Identity and access controls assume that users are humans operating within role-based sessions—not autonomous agents that require contextual, least-privilege permissions for every tool. This approach worked when AI was contained and deterministic, but now it's breaking down.

Today, AI agents discover tools dynamically via Model Context Protocol (MCP) servers and tool catalogs, share session state and persistent memory, and invoke other agents via standardized protocols such as MCP and agent-to-agent (A2A) interfaces. The result is a structural mismatch between architectures designed for human-driven workflows and the demands of autonomous, multi-agent systems.

Agentic AI raises the complexity bar well beyond what existing enterprise platforms were designed to handle. These systems involve multiple models and agents that share persistent memory, coordinate via multistep orchestration workflows, and communicate directly with other agents rather than operating in isolation. Each agent may invoke APIs, execute generated code in sandboxed environments, query knowledge bases through vector and graph indexes, and pass context to downstream agents—all within a single user request. This requires an orchestration layer that can manage the workflow end to end, including tool calls, context handoffs, and execution controls across agents within an application. It also calls for a centralized registry and policy layer to govern agent access, runtime guardrails such as prompt-injection filtering and content safety controls, and end-to-end traceability across the full execution graph.

Memory management must be treated as a first-class infrastructure concern rather than an application afterthought. Agent behaviors evolve rapidly and nondeterministically, so the underlying infrastructure must support faster deployment cycles, canary rollouts, automated rollback on service level objective (SLO) regression, and continuous evaluation. To meet these new requirements, organizations are modernizing their existing platforms, converging around three architectural layers—each with direct implications for delivery, risk, and scale.

Application and orchestration layer

This is the command center of the agent-based system. It directs multistep workflows through orchestration engines that manage control flow, retries, time-outs, and parallel execution.

Source: Bain & Company

At the platform level, orchestration and runtime services should be shared, along with identity and policy enforcement, observability and audit tooling, and centralized registries and catalogs for agents, tools, and entitlements. Every request is routed to a specialized agent through an orchestration layer that manages workflow control and context handoffs. This layer also coordinates A2A communication through standardized protocols (such as MCP and A2A), maintaining shared context, session memory, and state (e.g., variables, task progress, decisions made) across interactions.

Agents are deployed as separate yet connected versioned services that can be scaled, updated, and rolled back independently. Each is logged in an agent registry and policy layer with defined capabilities, tool entitlements, and policy constraints. Tool and API abstractions normalize external capabilities as MCP servers with consistent schemas and invocation semantics, while a tool catalog with life cycle management governs what is available and to whom.

Agent skills are managed the same way: with a governed library of approved capabilities that can be reused across use cases. Built-in access controls, identity propagation for nonhuman principals, and audit tooling reinforce accountability across every interaction. On top of this foundation, applications implement the use case logic, including task-specific agents and workflows, domain tool adapters, user experience and approval patterns, and evaluation and success criteria that define what “good” looks like in production. A mature platform should provide reusable components for these patterns so teams can deliver faster without rebuilding the wheel each time.

Analytics and insight layer

This layer provides real-time visibility into agent execution via metrics, logs, and traces collected across agents, workflows, and infrastructure. Full reasoning-path traceability captures every step—from prompt to tool invocation to final output—enabling teams to audit and explain agent decisions. Integrated monitoring, token management, and alignment monitoring (i.e., detecting behavioral drift, hallucination patterns, and bias signals) make it easier to manage compliance. Live dashboards and anomaly detection help teams maintain visibility as A2A interactions and behaviors evolve in production, ensuring everything stays transparent and aligned with policy.

Source: Bain & Company

Data and knowledge layer

This layer serves as the data foundation for agentic systems, integrating structured and unstructured data via standardized interfaces. It unifies data across domains (spanning relational, vector, and graph stores) and gives agents consistent, governed access they can rely on. Schema and data contract governance enforce compatibility across producers and consumers, while a federated data catalog provides discoverability and lineage.

Real-time streaming pipelines complement batch processing to ensure agents operate on current data, not stale snapshots. Metadata is automatically captured, and key data governance controls (e.g., classification, masking, retention, cross-domain access) are built into the pipelines to help ensure everything is used responsibly.

Together, these layers transform AI architecture from a collection of independent components into a shared enterprise capability. Security and governance are embedded across layers rather than applied after deployment. Control points, auditability, and oversight become part of normal operations.

Source: Bain & Company

But architecture alone isn't enough. In the next post, we'll explore why governance and trust are the true foundation for scaling agentic AI—and why data quality is the make-or-break factor.

Explore this series

- Why Agentic AI Demands a New Architecture
- The Three Layers of an Agentic AI Platform
- Governance, Trust, and the Data Foundation
- From Roadmap to Reality: Phasing Agentic AI into Production

Published in April 2026